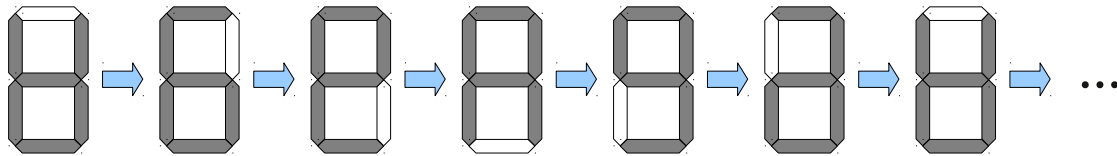


(20 pts) **Questão 1:** Crie um **programa** que apresente a seguinte animação nos 4 displays de 7 segmentos:



Os displays 1, 2, 3 e 4 são habilitados através dos terminais 2, 5, 3 e 1 da porta E, respectivamente.

Os segmentos são ligados com "1" e são acionados pelos bits da porta C, partindo do segmento superior em sentido horário: 0, 1, 2, 3, 4 e 5.

Utilize um timer para garantir tempo de 1 segundo entre as mudanças. Pode utilizar o *timer.h*.

```
#include "basico.h"
#include "config.h"
#include "timer.h"

void main(void) interrupt 0{
    char led;
    int i;
    InicializaTimer();
    TRISE = 0x00; //habilitando como saída os displays
    PORTE = 0xFF; //ligando todos os displays
    TRISC = 0x00; //habilitando como saída os leds
    for(;;){ //loop infinito
        for(i = 0; i<1000; i++){ //fazendo loop pra dar um tempo de 1 ms;
            ResetaTimer(1000);
            AguardaTimer();
        }
        //desligando/ligando todos os leds
        PORTC = 0x00; //PORTC = 0xFF;
        //liga/desliga o próximo led.
        BitSet(PORTC, led); //BitClr(PORTC, led);
        led++;
        If(led >=6){
            led = 0;
        }
    }
}
```

(20 pts) **Questão 2:** Na porta D estão ligadas 8 teclas, uma em cada terminal do microcontrolador. Faça uma função que realiza o debounce das teclas e retorna qual está pressionada: 1 para a tecla no terminal 0, 2 para a tecla no terminal 1, e assim até a última tecla. A rotina de debounce deve testar dez vezes se a saída está estável antes de retornar a chave pressionada.

```
char debounce(void){
```

```

char tecla = 0; //sem ninguem pressionado
char novatecla;
char tempo = 10;
char i;
//configura como entrada, não é necessário pois em geral
//a configuração fica na rotina principal
TRISD = 0xFF;
//enquanto tempo não esgotar espera estabilizar
while(tempo){
    novatecla = PORTD;
    //testa se está estável
    if (novatecla != tecla){
        tecla = novatecla;
        tempo = 10;
    } else {
        tempo--;
    }
}
//deste ponto em diante tecla já está estável
for(i = 0; i<8;i++){
    if(BitTst(tecla,i){
        return i+1;
    }
}
return 0; //sem tecla pressionada
}

```

(10 pts) **Questão 3:** O watchdog é um circuito eletrônico utilizado para melhorar a segurança dos sistemas , evitando que estes fiquem presos em loops infinitos. Para configurá-lo devemos atuar no registro CONFIG2H que é mostrado abaixo junto com suas opções. Faça sua **configuração** de modo que ele esteja habilitado e configurado para trabalhar com um postscaler de 1 para 8192. Apresente o resultado em decimal, hexadecimal e binário. Lembre-se que os registros especiais só podem ser acessados através das diretivas code ___ at ___ = ___. Por exemplo:

```

code char at 0x123456 REGISTRO = 0x3A;
//em binário 0b00111010
//em decimal 58

```

bit 7-5
bit 4-1

Unimplemented: Read as '0'

WDTPS3:WDTPS0: Watchdog Timer Postscale

1111 = 1:32,768
1110 = 1:16,384
1101 = 1:8,192
1100 = 1:4,096
1011 = 1:2,048
1010 = 1:1,024
1001 = 1:512
1000 = 1:256
0111 = 1:128
0110 = 1:64
0101 = 1:32
0100 = 1:16
0011 = 1:8
0010 = 1:4
0001 = 1:2
0000 = 1:1

WDTEN: Watchdog Timer Enable bit
1 = WDT enabled
0 = WDT disabled (control is placed on the SWD)

bit 0

REGISTER 25-4: CONFIG2H: CONFIGURATION REGISTER 2 HIGH (BYTE ADDRESS 300003h)

U-0	U-0	U-0	R/P-1	R/P-1	R/P-1	R/P-1	R/P-1
—	—	—	WDTPS3	WDTPS2	WDTPS1	WDTPS0	WDTEN
bit 7			bit 0				

(20 pts) Questão 4: Crie um **programa**, cíclico, que lê o valor de um potenciômetro ligado à entrada analógica e apresenta esse valor no display de 7 segmentos. Fazer a atualização do display de 7 segmentos a cada 5 ms, utilizando a interrupção do timer. Pode-se utilizar as funções do *timer.h*, *disp7seg.h* e *adc.h*.

OBS: Para ligar a interrupção do timer é necessário ligar os bits 5, 6 e 7 do registro INTCON e desligar o bit 7 do registro RCON. O bit 2 do INTCON é responsável por indicar se aconteceu ou não overflow no timer.

```
#include "basico.h"
#include "config.h"
#include "timer.h"
#include "disp7seg.h"
#include "adc.h"

//interrupção para reiniciar o timer e atualizar o timer
void interrupcao(void) interrupt 1{
    if(BitTst(INTCON,2){
        ResetaTimer(5000);
        AtualizaDisplay();
    }
}

void main(void) interrupt 0{
    unsigned int valor;
    InicializaTimer();
    InicializaDisplays();
    InicializaADC();
    BitSet(INTCON,5);
    BitSet(INTCON,6);
    BitSet(INTCON,7);
    BitClr(RCON,7);
    for(;;){ //loop infinito
        valor = LeValorAD();
        MudaDigito((valor / 1000)%10, 3);
        MudaDigito((valor / 100 )%10, 2);
        MudaDigito((valor / 10  )%10, 1);
        MudaDigito((valor          )%10, 0);
    }
}
```

(30 pts) Questão 5: Faça um programa que receba comandos via serial. Quando chegar a palavra "nome" pela serial, o programa deve escrever no LCD o nome do aluno. Lembre-se: a função RecebeSerial() retorna valor 0 (zero) quando não há nada a receber na serial; a palavra "nome" é composta de 4 caracteres que devem ser recebidos em sequência; não é possível ter certeza de quando um caracter será recebido, portanto o programa deve ficar esperando constantemente. Pode-se utilizar as funções do *lcd.h* e *serial.h*.

```

#include "basico.h"
#include "config.h"
#include "serial.h"
#include "lcd.h"

void main(void) interrupt 0{
    char qt_letras; //conta quantas letras corretas já chegaram
    char letra;
    InicializaSerial();
    InicializaLCD();
    qt_letras = 0;
    letra = 0;
    for(;;){ //loop infinito
        //aguarda chegar uma letra
        while(letra == 0){
            letra = RecebeSerial();
        }
        //testa se chegou 'n'
        if (letra == 'n'){
            qt_letra=1; //já chegou uma letra certa
        }
        //testa se a letra é 'o' e se já chegou 'n' antes
        if ((letra == 'o') && (qt_letra == 1)){
            qt_letra=2; //já chegaram duas letras certa
        } else {
            qt_letras = 0; //chegou uma errada tem que reiniciar
        }
        //testa se a letra é 'm' e se já chegou 'n' e 'o' antes
        if ((letra == 'm') && (qt_letra == 2)){
            qt_letra=3; //já chegaram três letras certa
        } else {
            qt_letras = 0; //chegou uma errada tem que reiniciar
        }
        //testa se a letra é 'e' e se já chegou 'n', 'o' e 'm' antes
        if ((letra == 'e') && (qt_letra == 3)){
            qt_letra=4; //já chegaram quatro letras certa
        } else {
            qt_letras = 0; //chegou uma errada tem que reiniciar
        }
        //se chegou a palavra "nome" via serial envia o nome pro LCD
        if(qt_letras == 4){
            EnviaDados("M");
            EnviaDados("a");
            EnviaDados("x");
            qt_letras = 0;
        }
    }
}

```

----- Funções disponíveis

//timer.h

```
void InicializaTimer(void);  
void ResetaTimer(unsigned int);  
void AguardaTimer(void);
```

//lcd.h

```
void InicializaLCD(void);  
void EnviaDados(unsigned char val);
```

//disp7seg.h

```
void InicializaDisplays(void);  
void MudaDigito(char val, char pos);  
void AtualizaDisplay(void);
```

//adc.h

```
void InicializaADC(void);  
unsigned int LeValorAD(void);
```

//serial.h

```
void InicializaSerial(void);  
char RecebeSerial(void);
```